

LAB EXPERIMENTS

V.RAKSHMITHA

192465052

EXP 1:

AIM: To implement the simple substitution technique named Caesar cipher using C language.

ALGORITHM:

STEP-1: Read the plain text from the user.

STEP-2: Read the key value from the user.

STEP-3: If the key is positive then encrypt the text by adding the key with each character in the plain text.

STEP-4: Else subtract the key from the plain text.

STEP-5: Display the cipher text obtained above.

PROGRAM:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char text[100], encrypt[100], decrypt[100];
```

```
    int key, i;
```

```
    printf("Enter the plain text:");
```

```
    scanf("%s", text);
```

```
    printf("Enter the key value:");
```

```
    scanf("%d", &key);
```

```
    // Encryption
```

```

for(i = 0; text[i] != '\0'; i++)
{
    if(text[i] >= 'a' && text[i] <= 'z')
        encrypt[i] = ((text[i] - 'a' + key) % 26) + 'a';
    else if(text[i] >= 'A' && text[i] <= 'Z')
        encrypt[i] = ((text[i] - 'A' + key) % 26) + 'A';
    else
        encrypt[i] = text[i];
}
encrypt[i] = '\0';

// Decryption
for(i = 0; encrypt[i] != '\0'; i++)
{
    if(encrypt[i] >= 'a' && encrypt[i] <= 'z')
        decrypt[i] = ((encrypt[i] - 'a' - key + 26) % 26) + 'a';
    else if(encrypt[i] >= 'A' && encrypt[i] <= 'Z')
        decrypt[i] = ((encrypt[i] - 'A' - key + 26) % 26) + 'A';
    else
        decrypt[i] = encrypt[i];
}
decrypt[i] = '\0';

printf("\n\nPLAIN TEXT: %s", text);
printf("\n\nENCRYPTED TEXT: %s", encrypt);
printf("\n\nAFTER DECRYPTION : %s", decrypt);

return 0;
}

```

OUTPUT:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char text[100], encrypt[100], decrypt[100];
    int key, i;

    printf("Enter the plain text:");
    scanf("%s", text);

    printf("Enter the key value:");
    scanf("%d", &key);

    // Encryption
    for(i = 0; text[i] != '\0'; i++)
    {
        if(text[i] >= 'a' && text[i] <= 'z')
            encrypt[i] = ((text[i] - 'a' + key) % 26) + 'a';
        else if(text[i] >= 'A' && text[i] <= 'Z')
            encrypt[i] = ((text[i] - 'A' + key) % 26) + 'A';
        else
            continue;
    }

    printf("PLAIN TEXT: %s\n", text);
    printf("ENCRYPTED TEXT: %s\n", encrypt);
    printf("AFTER DECRYPTION : %s\n", decrypt);

    return 0;
}
```

Enter the plain text:hello
Enter the key value:3

PLAIN TEXT: hello
ENCRYPTED TEXT: khor
AFTER DECRYPTION : hello

=== Code Execution Successful ===

EXPERIMENT 2:

AIM: To write a C program to implement the Playfair Substitution technique.

ALGORITHM:

STEP-1: Read the plain text from the user.

STEP-2: Read the keyword from the user.

STEP-3: Arrange the keyword without duplicates in a 5*5 matrix in the row order and fill the remaining cells with missed out letters in alphabetical order. Note that „i“ and „j“ takes the same cell.

STEP-4: Group the plain text in pairs and match the corresponding corner letters by forming a rectangular grid.

STEP-5: Display the obtained cipher text.

PROGRAM:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
char matrix[5][5];
```

```
void generateMatrix(char key[])
```

```
{
```

```
    int used[26] = {0};
```

```
    int i, j = 0, k = 0;
```

```
    used['J' - 'A'] = 1;
```

```
    for(i = 0; key[i] != '\0'; i++)
```

```
    {
```

```
        char ch = toupper(key[i]);
```

```
        if(ch == 'J') ch = 'I';
```

```
        if(ch >= 'A' && ch <= 'Z' && !used[ch - 'A'])
```

```
        {
```

```
            matrix[j][k++] = ch;
```

```
            used[ch - 'A'] = 1;
```

```
            if(k == 5)
```

```
            {
```

```
                k = 0;
```

```
                j++;
```

```
            }
```

```
        }
```

```
    }
```

```
    for(i = 0; i < 26; i++)
```

```
    {
```

```
        if(!used[i])
```

```

    {
        matrix[j][k++] = i + 'A';
        if(k == 5)
        {
            k = 0;
            j++;
        }
    }
}

void findPosition(char ch, int *row, int *col)
{
    if(ch == 'J') ch = 'I';

    for(int i = 0; i < 5; i++)
    {
        for(int j = 0; j < 5; j++)
        {
            if(matrix[i][j] == ch)
            {
                *row = i;
                *col = j;
                return;
            }
        }
    }
}

void prepareText(char text[], char result[])
{

```

```

int i, j = 0;
char temp[100];
for(i = 0; text[i] != '\0'; i++)
{
    if(isalpha(text[i]))
    {
        temp[j++] = toupper(text[i]);
    }
}
temp[j] = '\0';
j = 0;
for(i = 0; temp[i] != '\0'; i++)
{
    result[j++] = temp[i];
    if(temp[i] == temp[i + 1])
        result[j++] = 'X';
}
if(j % 2 != 0)
    result[j++] = 'X';
result[j] = '\0';
}

void encrypt(char text[])
{
    int i, r1, c1, r2, c2;
    printf("\nEncrypted Text: ");
    for(i = 0; text[i] != '\0'; i += 2)
    {
        findPosition(text[i], &r1, &c1);
        findPosition(text[i + 1], &r2, &c2);
    }
}

```

```

    if(r1 == r2)
    {
        printf("%c%c", matrix[r1][(c1 + 1) % 5], matrix[r2][(c2 + 1) % 5]);
    }
    else if(c1 == c2)
    {
        printf("%c%c", matrix[(r1 + 1) % 5][c1], matrix[(r2 + 1) % 5][c2]);
    }
    else
    {
        printf("%c%c", matrix[r1][c2], matrix[r2][c1]);
    }
}

int main()
{
    char key[50], plain[100], prepared[100];

    int i, j;

    printf("Enter the key: ");

    scanf("%s", key);

    printf("Enter the plain text: ");

    scanf("%s", plain);

    generateMatrix(key);

    printf("\nPlayfair Matrix:\n");

    for(i = 0; i < 5; i++)
    {
        for(j = 0; j < 5; j++)
            printf("%c ", matrix[i][j]);
    }
}

```

```

        printf("\n");
    }

    prepareText(plain, prepared);

    printf("\nPrepared Text: %s\n", prepared);

    encrypt(prepared);

    return 0;
}

```

OUTPUT:

```

main.c
120 int i, j;
121
122 printf("Enter the key: ");
123 scanf("%s", key);
124
125 printf("Enter the plain text: ");
126 scanf("%s", plain);
127
128 generateMatrix(key);
129
130 printf("\nPlayfair Matrix:\n");
131 for(i = 0; i < 5; i++)
132 {
133     for(j = 0; j < 5; j++)
134         printf("%c ", matrix[i][j]);
135     printf("\n");
136 }
137
138 prepareText(plain, prepared);
139
140 printf("\nPrepared Text: %s\n", prepared);
141

```

Output

```

Enter the key: HELLO
Enter the plain text: cse

Playfair Matrix:
H E L O A
B C D F G
I K M N P
Q R S T U
V W X Y Z

Prepared Text: CSEX

Encrypted Text: DRLW

=== Code Execution Successful ===

```

EXP 3:

AIM: To write a C program to implement the hill cipher substitution techniques.

ALGORITHM:

STEP-1: Read the plain text and key from the user.

STEP-2: Split the plain text into groups of length three.

STEP-3: Arrange the keyword in a 3*3 matrix.

STEP-4: Multiply the two matrices to obtain the cipher text of length three.

STEP-5: Combine all these groups to get the complete cipher text.

PROGRAM:

```
#include<stdio.h>
```

```

#include<conio.h>
#include<string.h>
int main()
{
    unsigned int a[3][3]={ {6,24,1},{13,16,10},{20,17,15}};
    unsigned int b[3][3]={ {8,5,10},{21,8,21},{21,12,8}};
    unsigned int c[20],d[20];
    char msg[20];
    int i,j,t;
    clrscr();
    printf("Enter Plain Text (3 letters): ");
    scanf("%s",msg);
    for(i=0;i<3;i++)
        c[i]=msg[i]-'A';
    for(i=0;i<3;i++)
    {
        t=0;
        for(j=0;j<3;j++)
            t+=a[i][j]*c[j];
        d[i]=t%26;
    }
    printf("\nEncrypted Cipher Text : ");
    for(i=0;i<3;i++)
        printf("%c",d[i]+'A');
    for(i=0;i<3;i++)
    {
        t=0;
        for(j=0;j<3;j++)
            t+=b[i][j]*d[j];

```

```

        c[i]=t%26;

    }

    printf("\nDecrypted Cipher Text : ");

    for(i=0;i<3;i++)

        printf("%c",c[i]+'A');

    getch();

    return 0;

}

```

OUTPUT:

The screenshot shows a C program in a code editor on the left and its output in a terminal window on the right. The code defines a 3x3 key matrix and processes a 3-letter plain text. The output shows the encrypted text 'BRZ' and the decrypted text 'FGJ'.

```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      unsigned int a[3][3]={6,24,1},{13,16,10},{20,17,15}};
7      unsigned int b[3][3]={8,5,10},{21,8,21},{21,12,8}};
8      unsigned int c[20], d[20];
9      char msg[20];
10     int i, j, t;
11
12     printf("Enter Plain Text (3 letters): ");
13     scanf("%s", msg);
14
15     for(i=0;i<3;i++)
16     {
17         c[i]=msg[i]-'A';
18     }
19
20     for(i=0;i<3;i++)
21     {
22         t=0;
23         for(j=0;j<3;j++)
24             t+=a[i][j]*c[j];
25     }
26
27     for(i=0;i<3;i++)
28     {
29         d[i]=t%26+'A';
30         t=0;
31     }
32
33     printf("\nEncrypted Cipher Text : ");
34     for(i=0;i<3;i++)
35         printf("%c",d[i]+'A');
36
37     printf("\nDecrypted Cipher Text : ");
38     for(i=0;i<3;i++)
39         printf("%c",c[i]+'A');
40
41     getch();
42     return 0;
43 }

```

Enter Plain Text (3 letters): FGJ

Encrypted Cipher Text: BRZ

Decrypted Cipher Text: FGJ

=== Code Execution Successful ===

EXP 4:

AIM: To implement the Vigenere Cipher substitution technique using C program.

ALGORITHM:

STEP-1: Arrange the alphabets in row and column of a 26*26 matrix.

STEP-2: Circulate the alphabets in each row to position left such that the first letter is attached to last.

STEP-3: Repeat this process for all 26 rows and construct the final key matrix.

STEP-4: The keyword and the plain text is read from the user.

STEP-5: The characters in the keyword are repeated sequentially so as to match with that of the plain text.

STEP-6: Pick the first letter of the plain text and that of the keyword as the row indices and column indices respectively.

STEP-7: The junction character where these two meet forms the cipher character.

STEP-8: Repeat the above steps to generate the entire cipher text.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

void encipher();
void decipher();

int main()
{
    int choice;
    while(1)
    {
        printf("\n1. Encrypt Text");
        printf("\n2. Decrypt Text");
        printf("\n3. Exit");
        printf("\n\nEnter Your Choice: ");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                encipher();
                break;
            case 2:
                decipher();
                break;
```

```

        case 3:
            return 0;
default:
    printf("\nInvalid Choice!\n");
    }
    }
}

void encipher()
{
    char input[100], key[100];
    int i, j;
    printf("\nEnter Plain Text: ");
    scanf("%s", input);
    printf("Enter Key: ");
    scanf("%s", key);
    printf("Encrypted Cipher Text: ");
    for(i=0, j=0; i<strlen(input); i++, j++)
    {
        if(j==strlen(key))
            j=0;
        printf("%c", ((toupper(input[i])-'A') + (toupper(key[j])-'A')) % 26 + 'A');
    }
    printf("\n");
}

void decipher()
{
    char input[100], key[100];
    int i, j, value;

```

```

printf("\nEnter Cipher Text: ");

scanf("%s", input);

printf("Enter Key: ");

scanf("%s", key);

printf("Decrypted Plain Text: ");

for(i=0, j=0; i<strlen(input); i++, j++)

{

    if(j==strlen(key))

        j=0;

    value = (toupper(input[i])-'A') - (toupper(key[j])-'A');

    if(value < 0)

        value += 26;

    printf("%c", value + 'A');

}

printf("\n");
}

```

OUTPUT:



The screenshot shows a C program running in a terminal window. The program is a Caesar cipher implementation. It starts with a menu: 1. Encrypt Text, 2. Decrypt Text, 3. Exit. The user enters '2' to choose decryption. Then, it prompts for 'Enter Cipher Text:' where 'XFOPVEFRNHR' is entered, and 'Enter Key:' where 'EMON' is entered. The output is 'Decrypted Plain Text: TTACRSREJVD'. The menu is shown again, and the user enters '1' to choose encryption. The program then prompts for 'Enter Your Choice: |'.

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

void encipher();
void decipher();

int main()
{
    int choice;

    while(1)
    {
        printf("\n1. Encrypt Text");
        printf("\n2. Decrypt Text");
        printf("\n3. Exit");

        printf("\n\nEnter Your Choice: ");
        scanf("%d",&choice);

        switch(choice)

```

EXP 5-

AIM: To write a C program to implement the rail fence transposition technique.

ALGORITHM:

STEP-1: Read the Plain text.

STEP-2: Arrange the plain text in row columnar matrix format.

STEP-3: Now read the keyword depending on the number of columns of the plain text.

STEP-4: Arrange the characters of the keyword in sorted order and the corresponding columns of the plain text.

STEP-5: Read the characters row wise or column wise in the former order to get the cipher text.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

int main()
{
    int i, j, k, l;
    char a[100], c[100], d[100];
    printf("\n\t\tRAIL FENCE TECHNIQUE\n");
    printf("Enter the input string: ");
    scanf("%s", a);
    l = strlen(a);
    /* Encryption */
    for(i=0, j=0; i<l; i++)
    {
        if(i%2==0)
            c[j++] = a[i];
    }
}
```

```

for(i=0; i<l; i++)
{
    if(i%2==1)
        c[j++] = a[i];
}
c[j] = '\0';
printf("\nCipher Text: %s\n", c);

/* Decryption */
if(l%2==0)
    k=l/2;
else
    k=(l/2)+1;
for(i=0,j=0;i<k;i++)
{
    d[j]=c[i];
    j+=2;
}

for(i=k,j=1;i<l;i++)
{
    d[j]=c[i];
    j+=2;
}

d[l]='\0';
printf("Decrypted Text: %s\n", d);
return 0;
}

```

OUTPUT:

```
#include <stdio.h>
#include <string.h>

int main()
{
    int i, j, k, l;
    char a[100], c[100], d[100];

    printf("\n\t\tRAIL FENCE TECHNIQUE\n");

    printf("Enter the input string: ");
    scanf("%s", a);

    l = strlen(a);

    /* Encryption */
    for(i=0, j=0; i<l; i++)
    {
        if(i%2==0)
            c[j++] = a[i];
    }

    printf("Cipher Text: HLOOLELWRD\n");
    printf("Decrypted Text: HELLOWORLD\n");

    return 0;
}
```

RAIL FENCE TECHNIQUE
Enter the input string: HELLOWORLD
Cipher Text: HLOOLELWRD
Decrypted Text: HELLOWORLD
=== Code Execution Successful ===

EXP 6-

AIM: To write a C program to implement Data Encryption Standard (DES) using C Language.

ALGORITHM:

STEP-1: Read the 64-bit plain text.

STEP-2: Split it into two 32-bit blocks and store it in two different arrays.

STEP-3: Perform XOR operation between these two arrays.

STEP-4: The output obtained is stored as the second 32-bit sequence and the original second 32-bit sequence forms the first part.

STEP-5: Thus the encrypted 64-bit cipher text is obtained in this way. Repeat the same process for the remaining plain text characters.

PROGRAM:

```
#include <stdio.h>

#include <string.h>

int main()
{
    char plain[65], key[33];

    char left[33], right[33], temp[33];
```

```

    int i;

    printf("Enter 64-bit Plain Text (0s and 1s): ");

    scanf("%64s", plain);

    printf("Enter 32-bit Key (0s and 1s): ");

    scanf("%32s", key);

/* XOR Right Half with Key */

    for(i = 0; i < 32; i++)

        temp[i] = ((right[i] - '0') ^ (key[i] - '0')) + '0';

    temp[32] = '\0';

/* Cipher = Right + (Left XOR temp) */

    printf("\nEncrypted 64-bit Cipher Text:\n");

    printf("%s", right);

    for(i = 0; i < 32; i++)

        printf("%c", ((left[i] - '0') ^ (temp[i] - '0')) + '0');

    printf("\n");

return 0;

}

```

OUTPUT:

```

#include <stdio.h>
#include <string.h>

int main()
{
    char plain[65], key[33];
    char left[33], right[33], temp[33];
    int i;

    printf("Enter 64-bit Plain Text (0s and 1s): ");
    scanf("%64s", plain);

    printf("Enter 32-bit Key (0s and 1s): ");
    scanf("%32s", key);

    for(i = 0; i < 32; i++)
    {
        left[i] = plain[i];
        right[i] = plain[i + 32];
    }
    left[32] = '\0';
    right[32] = '\0';

    Enter 64-bit Plain Text (0s and 1s):
    10101010101010101010101010101010111100001111000011110000

    Enter 32-bit Key (0s and 1s): 11110000111100001111000011110000

    Encrypted 64-bit Cipher Text:
    1111000011110000111100001111000010101010101010101010101010101010

    === Code Execution Successful ===

```